CS 344: Design and Analysis of Computer Algorithms **Rutgers: Spring 2026**

Divide and Conquer

Binary Search

Problem 1. A sequence of integers strictly increases and then strictly decreases, find the maximum element of the sequence in $O(\log n)$ time.

Problem 2. Describe an $O(\log n)$ time that either computes an index i such that $A[i] = i$ or correctly reports that no such index exists.

Suppose we know in advance that $A[1] > 0$. Describe an even faster algorithm that either computes an index i such that $A[i] = i$ or correctly reports that no such index exists. [Hint: This is really easy.]

Problem 3. Suppose you are given two sorted arrays $A[1..n]$ and $B[1..n]$ containing distinct integers. Describe a fast algorithm to find the median (meaning the nth smallest element) of the union $A \cup B$. For example, given the input

$$A[1..8] = [0, 1, 6, 9, 12, 13, 18, 20] \quad B[1..8] = [2, 4, 5, 8, 17, 19, 21, 23]$$

your algorithm should return the integer 9. [Hint: What can you learn by comparing one element of A with one element of B ?]

Problem 4. Now suppose you are given two sorted arrays $A[1..m]$ and $B[1..n]$ and an integer k. Describe a fast algorithm to find the k th smallest element in the union $A \cup B$. For example, given the input

$$A[1..8] = [0, 1, 6, 9, 12, 13, 18, 20] \quad B[1..5] = [2, 5, 7, 17, 19] \quad k = 6$$

your algorithm should return the integer 7.

Recursion

Problem 5. The recursive algorithm to compute Fibonacci numbers is given by

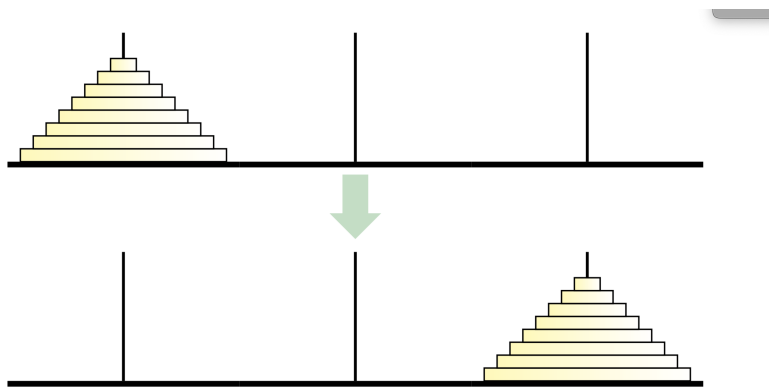
$$F(n) = \begin{cases} 0, & \text{if } n = 0, \\ 1, & \text{if } n = 1, \\ F(n-1) + F(n-2), & \text{if } n \geq 2. \end{cases}$$

Prove that the time complexity is

$$T(n) = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right).$$

Problem 6. The classical Tower of Hanoi consists of three pegs and n disks of distinct sizes. The goal is to move all disks from the source peg to the destination peg while following these rules:

- Only one disk can be moved at a time.
- A larger disk cannot be placed on a smaller disk.
- Only the top disk of a peg can be moved.



The classical recursive algorithm follows the strategy:

- Move the top $n-1$ disks from peg x to peg y via peg z.
- Move disk n from peg x to peg z.
- Move the $n-1$ disks from peg y to peg z via peg x.

This results in the recurrence relation:

$$T(n) = 2T(n-1) + 1$$

which solves to $T(n) = 2^n - 1$ moves.

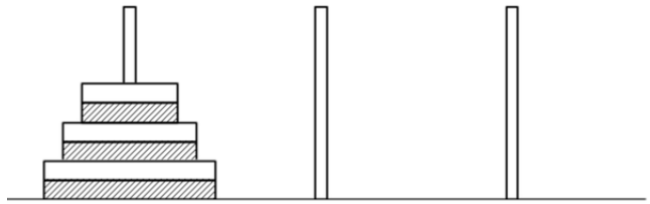
Prove that the original recursive Tower of Hanoi algorithm performs exactly the same sequence of moves-the same disks, to and from the same pegs, in the same order-as each of the following non-recursive algorithms. The pegs are labeled 0,1 , and 2 , and our problem is to move a stack of n disks from peg 0 to peg 2.

- If n is even, swap pegs 1 and 2 . At the i th step, make the only legal move that avoids peg i mod 3. If there is no legal move, then all disks are on peg i mod 3, and the puzzle is solved.
- For the first move, move disk 1 to peg 1 if n is even and to peg 2 if n is odd. Then repeatedly make the only legal move that involves a different disk from the previous move. If no such move exists, the puzzle is solved.
- Pretend that disks $n+1, n+2$, and $n+3$ are at the bottom of pegs 0,1 , and 2 , respectively. Repeatedly make the only legal move that satisfies the following constraints, until no such move is possible.

Following are some variation of the Tower of Hanoi problem:

1. If all moves must be between adjacent pegs, what is the number of movies required? That is you cannot directive move from x to z for instance.

2. Now, consider a variation where for each disk size k, there are two identical disks. The goal remains to move all disks from peg x to peg z while following the standard Tower of Hanoi rules.



   - If we ignore the fact that disks of the same size are identical and treat them as distinct disks, we can simply apply the standard Tower of Hanoi solution to $2n$ disks. What is the recurrence relation and the resulting number of moves?
   - If we move both disks of the same size simultaneously while using standard Hanoi's algorithm, we are close to the solution. This strategy may flip the order of some equal size disks. Repeat this again to restore the order of disks. How many moves does this take?
   - **(Extra Credit)**: What is the optimal strategy to minimize the number of moves while preserving the original order of the disks?

3. Now we consider a variation with k pegs. Let $T_k(n)$ denote the minimum number of moves required to transfer n disks using k pegs. The classical problem with three pegs follows the recurrence:

$$T_3(n) = 2T_3(n-1) + 1,$$

   which solves to $T_3(n) = 2^n - 1$.

   For four pegs, the Frame–Stewart algorithm follows these steps:

   - Move the top k disks to an auxiliary peg using four pegs.
   - Move the remaining $n-k$ disks directly to the destination using three pegs.
   - Move the top k disks from the auxiliary peg to the destination.

   Write the recurrence relation for $T_k(n)$ for arbitrary k.

   Extra Credit: Find the optimal k and solve the recurrence.

4.

Divide and Conquer

Problem 7. Standard merge sort works as follows:

- The array is split into two equal halves of size $n/2$.
- Each half is recursively sorted.
- The two sorted halves are merged using the two-pointer technique in $O(n)$.

Consider the following variants:

- Instead of splitting into equal halves, the array is split into two parts of size $2n/3$ and $n/3$. What is the recurrence relation and time complexity?

- Instead of two parts, the array is split into three equal parts of size $n/3$. What is the recurrence relation and time complexity?
- The array is split into k equal parts of size n/k. How does the choice of k affect time complexity?
- The array is split into two parts of sizes k and $n-k$. How does the choice of k affect time complexity?

Problem 8. Find a recursive algorithm to compute a^n with $O(\log n)$ multiplications.

Problem 9. An inversion in an array $A[1..n]$ is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. The number of inversions in an n-element array is between 0 (if the array is sorted) and $\binom{n}{2}$ (if the array is sorted backward). Describe and analyze an algorithm to count the number of inversions in an n-element array in $O(n \log n)$ time. [Hint: Modify mergesort.]

Problem 10. Design a divide-and-conquer algorithm to find the largest element in an array

Problem 11. Write a divide-and-conquer algorithm that finds the maximum difference between any two elements of a given array of n numbers (not necessarily distinct) in $O(n)$ time. For example, on input $A = [4.5, 10, -2, \pi, -7.115]$, your algorithm should return 17.115.

Problem 12. Given an integer array nums, design a divide-and-conquer algorithm to find the contiguous subarray with the largest sum.

- First, design an $O(n \log n)$ algorithm using the divide-and-conquer approach.
- Then, solve a more general problem that enables an optimized $O(n)$ divide-and-conquer solution.

Problem 13. Given an integer array $A[1 : n]$, where the array is circular (i.e., elements wrap around from the end to the beginning), design an algorithm to compute the maximum sum of any contiguous subarray.

- Show how this problem can be solved by considering two cases:
  1. The maximum subarray lies entirely within the linear array.
  2. The maximum subarray wraps around, which can be computed using the total sum of the array and the minimum subarray sum.
- Prove that the circular maximum subarray sum is given by:

$$\max(S_{\max}, \text{sum}(A) - S_{\min})$$

  where $S_{\max}$ is the standard maximum subarray sum and $S_{\min}$ is the minimum subarray sum.
- Design an $O(n)$ algorithm to compute the result.

Problem 14. Given an integer array, design a divide-and-conquer algorithm to find the longest contiguous subarray that is strictly increasing.

- First, design an $O(n \log n)$ algorithm using the divide-and-conquer approach.
- Then, solve a more general problem that enables an optimized $O(n)$ divide-and-conquer solution.

Problem 15. Given an array $A[1 : n]$, where each element represents an interval, design a divide-and-conquer algorithm to compute the maximum overlap between any two intervals.

- Divide the array into two halves and recursively compute the maximum overlap in each half.
- Merge the results efficiently using a two-pointer technique to find the maximum overlap across both halves.

- Return the maximum of the overlaps found in the left half, right half, and across the two halves.
- Analyze the time complexity and show that the algorithm runs in $O(n \log n)$.

Problem 16. Given an integer array where a majority element appears more than $\lfloor n/2 \rfloor$ times, design a divide-and-conquer algorithm to find this element efficiently in $O(n)$ time.

- First, design an $O(n \log n)$ algorithm using the divide-and-conquer approach.
- Then, solve a more general problem that enables an optimized $O(n)$ divide-and-conquer solution.

Problem 17. A collection of n identification cards is given, each corresponding to a unique account. Some accounts may have multiple cards associated with them. Two cards are considered equivalent if they belong to the same account.

A verification system can check whether two given cards are equivalent. The goal is to determine whether more than $n/2$ of the cards belong to the same account.

- Design an algorithm that solves this problem using only $O(n \log n)$ equivalence checks.
- Justify its correctness and analyze its time complexity.

Problem 18. Here is a variation of Karatsuba's algorithm:

Given two n-digit numbers x and y, assume $n = 3^k$ for integer k. Define:

$$x = x_2 \cdot 10^{2n/3} + x_1 \cdot 10^{n/3} + x_0, \quad y = y_2 \cdot 10^{2n/3} + y_1 \cdot 10^{n/3} + y_0.$$

Now, compute the intermediate products:

$$\begin{aligned}
&w_1 = x_0 y_0, \quad w_2 = (x_0 + x_1 + x_2)(y_0 + y_1 + y_2), \\
&w_3 = (x_0 - x_1 + x_2)(y_0 - y_1 + y_2), \quad w_4 = (x_0 + 2x_1 + 4x_2)(y_0 + 2y_1 + 4y_2), \\
&w_5 = x_2 y_2.
\end{aligned}$$

Define:

$$\begin{aligned}
&z_1 = w_1, \quad z_2 = -\frac{w_1}{2} + w_2 - \frac{w_3}{3} - \frac{w_4}{6} + 2w_5, \\
&z_3 = -w_1 + \frac{w_2}{2} + \frac{w_3}{2} - w_5, \\
&z_4 = \frac{w_1}{2} - \frac{w_2}{2} - \frac{w_3}{6} + \frac{w_4}{6} - 2w_5, \quad z_5 = w_5.
\end{aligned}$$

- Design a divide-and-conquer algorithm to compute $x \cdot y$ using the z-values.
- Show that this method reduces the number of recursive multiplications compared to Karatsuba.
- Derive the asymptotic time complexity of this algorithm.

Remark: The formulas used in this algorithm arise from **polynomial interpolation**. A polynomial can be determined either by its coefficients or by its values at various points: A polynomial of degree d is uniquely determined by $d + 1$ values.

Problem 19. The Sumerians may have used the identity:

$$x \cdot y = \frac{x^2 + y^2 - (x-y)^2}{2}$$

to perform multiplication via squaring. Modern recursive methods improve on this.

- (a) Describe a variant of Karatsuba's algorithm that squares an n-digit number in $O(n^{\log_2 3})$ time by reducing to three squarings of $\lceil n/2 \rceil$-digit numbers.
- (b) Describe a recursive algorithm that squares an n-digit number in $O(n^{\log_3 6})$ time by reducing to six squarings of $\lceil n/3 \rceil$-digit numbers.
- (c) Describe a recursive algorithm that squares an n-digit number in $O(n^{\log_3 5})$ time by reducing to five squarings of $(n/3 + O(1))$-digit numbers. [Hint: Consider $(a+b+c)^2 + (a-b+c)^2$.]

Problem 20. Describe and analyze a variant of Karatsuba's algorithm that multiplies an m-digit number by an n-digit number ($n \geq m$) in $O(nm^{\log_2 3 - 1})$ time.

- Using the result from part (a), describe an algorithm to compute the decimal representation of 2^n in $O(n^{\log_2 3})$ time. (The standard digit-by-digit method takes $\Theta(n^2)$ time.)
- Design a divide-and-conquer algorithm to compute the decimal representation of an arbitrary n-bit binary number in $O(n^{\log_2 3})$ time.

Problem 21. The recursive Euclidean algorithm:

$$\text{EUCLIDGCD}(x, y) = \begin{cases} x, & \text{if } x = y, \\ \text{EuclidGCD}(x - y, y), & x > y, \\ \text{EuclidGCD}(x, y - x), & x < y. \end{cases}$$

Prove that:

- $\text{EuclidGCD}(x, y)$ is a divisor of x, y.
- Every divisor of x, y also divides $\text{EuclidGCD}(x, y)$.

Analyze its worst-case time complexity, assuming $O(\log x + \log y)$ for subtraction.

Now consider:

$$\text{FASTEUCLIDGCD}(x, y) = \begin{cases} x, & y = 0, \\ \text{FastEuclidGCD}(y, x \bmod y). \end{cases}$$

Prove correctness and analyze its worst-case time complexity, assuming $O(\log x \cdot \log y)$ for modulo.

Problem 22. Consdier the problem of computing the Fibonacci numbers:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

a) Write an algorithm to compute the Fibonacci number F_n with $O(n)$ additions.

Note that F_n grows exponentially and has $\Theta(n)$ digits which means that the additions operation $F_n + F_{n+1}$ actually takes $O(n)$ time. Compute the time complexity of the algorithm.

b) We start by writing the equations $F_1 = F_1$ and $F_2 = F_0 + F_1$ in matrix notation:

$$\begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

Similarly,

$$\begin{pmatrix} F_2 \\ F_3 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^2 \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

In general,

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \cdot \begin{pmatrix} F_0 \\ F_1 \end{pmatrix}$$

So, in order to compute F_n, it suffices to raise this 2×2 matrix, call it X, to the nth power.

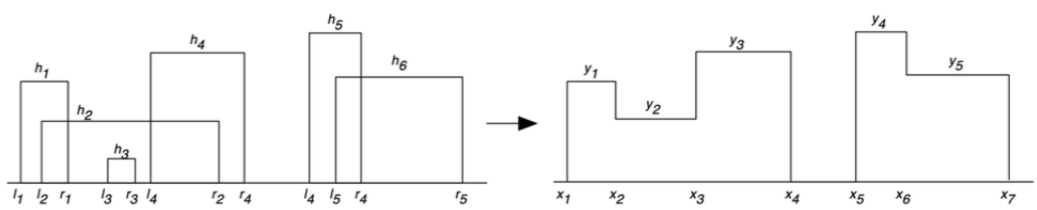
i) Show that $O(\log n)$ matrix multiplications suffice for computing X^n by repeated squaring. Note that two 2×2 matrices can be multiplied using 4 additions and 8 multiplications.

ii) Let $M(n)$ be the running time of an algorithm for multiplying n-bit numbers. Prove that the running time of this second algorithm is $O(M(n) \log n)$.

Problem 23. Given a list of buildings represented by their left x-coordinate, height, and right x-coordinate:

$$(l_1, h_1, r_1), (l_2, h_2, r_2), \ldots, (l_n, h_n, r_n)$$

where the list is sorted by increasing left x-coordinate, design an $O(n \log n)$ algorithm to compute the skyline.



The skyline should be output in the format:

$$x_1, y_1, x_2, y_2, x_3, y_3, \ldots$$

meaning that at x_1, the height is y_1, and this extends until x_2, where the height changes to y_2, and so on.

- Describe a divide-and-conquer approach to solve this problem efficiently.
- Justify the correctness of your algorithm and analyze its time complexity.

Problem 24. Suppose you have an algorithm that given as input a directed graph $G = (V, E)$, nodes $s, t \in V$, and an integer k, outputs whether the number of distinct shortest paths from s to t is at least k. Describe an algorithm that counts the number of distinct shortest $s - t$ paths in G. Does your algorithm run in polynomial time?